MMOT225: Bulut Bilişim
Hafta 5: Tekrar ve Genişletilmiş Docker Uygulamaları

Dr. Öğr. Üyesi Hasan Oğuz

İstanbul Okan Üniversitesi | Meslek Yüksekokulu

Genel Tekrar: Bulut Bilişim Temel Yaklaşımlar

1) Geleneksel (On-Premise)

- Sunucular kurum içinde.
- Donanım satın alma + bakım sorumluluğu.
- Yüksek maliyet, düşük esneklik.

2) Sanallaştırma

- 1 fiziksel makine → çoklu VM.
- Kaynak paylaşımı (CPU, RAM, Disk).
- Daha verimli donanım kullanımı.

3) Bulut Bilişim

- Kaynaklar internet üzerinden.
- İsteğe bağlı başlatma (dakikalar içinde).
- "Kullandığın kadar öde" modeli.

Günlük Hayat

- Drive, Netflix, e-Devlet vb.
- Veri merkezi altyapısı arka planda.

Temel Mantık

Sanallaştırma = **Donanımı verimli bölmek**
Bulut = **Bu kaynakları internetten hizmet olarak almak**

Genel Tekrar: Sanallaştırma ve Temel Altyapı

1. Sanallaştırma ve Hipervizör

- **Sanal Makine (VM):** Donanımı paylaşan, kendi OS'ine sahip izole çalışma ortamı.
- **Hipervizör:** Kaynakları (CPU, RAM, Disk) paylaştıran kontrol yazılımı.
  - **Type 1 (Bare Metal):** Doğrudan donanıma kurulur.
  - **Type 2:** İşletim sistemi üzerine kurulur.
- **İzolasyon:** Bir VM'in çökmesi diğerlerini ve Host'u etkilemez.

2. Kazanımlar ve Lab Özeti

- **Avantajlar:** Daha az fiziksel sunucu, düşük enerji maliyeti, hızlı ölçekleme.
- **Sınırlamalar:** Host donanımı arızalanırsa tüm VM'ler kapanır.
- **Snapshot & Altyapı:** Riskli işlemler öncesi sistemin dondurulması. Bu izole altyapı, **Docker ve Kubernetes** gibi bulut sistemlerinin temelini oluşturur.

Genel Tekrar: Altyapı ve Sanal Diskler

Sanal Makine Anatomisi ve Snapshot

- **VM = Dosyalar Bütünü:** Sanal makine fiziksel bir donanım değil, konfigürasyon (`.vmx`/`.vbox`) ve sanal disk (`.vmdk`/`.vdi`) dosyalarından ibarettir.
- **Snapshot:** Sistemin belirli bir andaki durumunu dondurur. Bir **yedekleme (backup) yöntemi değildir**, riskli işlemler öncesi hızlı geri dönüş noktasıdır.

Thick Provisioning (Kalın Tahsis)

- Disk kapasitesi **peşin** ayrılır.
- Yüksek I/O performansı sağlar.
- Kritik veritabanları için idealdir.

Thin Provisioning (İnce Tahsis)

- Kapasite **kullandıkça** (dinamik) büyür.
- Over-provisioning (aşırı tahsis) imkanı verir.
- Geliştirme/test ortamları için idealdir.

Genel Tekrar: Sanal Ağ Modelleri Karşılaştırması

Sanal makinelerin fiziksel dünyayla nasıl haberleştiğini belirleyen 3 temel model:

Ağ Modeli	IP Alma Mantığı	Kullanım Amacı ve Özellikleri
1. NAT	Hipervizör iç ağ yaratır, dışarı Host'un IP'si ile çıkar.	**Varsayılan moddur.** VM internete girer ama dışarıdan doğrudan ulaşılamaz.
2. Bridged	Doğrudan fiziksel ortamdaki modemden IP alır.	VM fiziksel ağın tam üyesi olur. Dışarıdan erişilebilir (Sunucu için).
3. Host-Only	İnternet kapalıdır. Sadece hipervizör içinde IP dağıtılır.	**Tam izolasyon.** Zararlı yazılım analizi veya kapalı devre testler için.

Genel Tekrar: Bulut Karakteristiği ve Konteynerler

Bulutun 5 Temel Özelliği (NIST)

1. **On-Demand:** Anında self-servis kaynak.
2. **Geniş Ağ Erişimi:** Her cihazdan ulaşılabilirlik.
3. **Kaynak Havuzu:** Fiziksel kaynakların ortak kullanımı.
4. **Hızlı Esneklik:** Otomatik büyüme/küçülme.
5. **Ölçülebilir Hizmet:** Kullandığın kadar öde mantığı.

Neden Konteyner (Docker)?

- **Ortak Kernel:** Sanal makineler (VM) gibi her seferinde ağır bir misafir OS çalıştırmaz; Host OS'in çekirdeğini paylaşır.
- **Hız ve Boyut:** Gigabaytlar yerine Megabaytlar boyutundadır. Saniyeler içinde başlar.
- **Taşınabilirlik:** "Benim makinemde çalışıyordu" problemini çözer.

Geçen Haftanın Laboratuvarı: Kurulum ve İlk Çalıştırma

Geçen hafta Ubuntu sanal makinemiz üzerine Docker Engine kurmuş ve sistemin çalışıp çalışmadığını test etmiştik.

Kurulum Komutları Özeti

`$ sudo apt update` *(Paketleri listelerini günceller)*
`$ sudo apt install docker.io -y` *(Docker'ı kurar)*
`$ sudo systemctl enable --now docker` *(Servisi aktif eder ve başlatır)*

İlk İmaj ve Konteyner (`hello-world`)

```
$ sudo docker run hello-world
```

Bu komutu çalıştırdığımızda sistemde gerçekleşen zincirleme reaksiyon:

1. Docker lokalde imajı aradı, bulamayınca **Docker Hub**'dan indirdi (pull).
2. İndirilen imajdan çalışan bir kopya (**Konteyner**) oluşturdu.
3. Konteyner içindeki basit scripti çalıştırıp ekrana mesaj yazdırdı ve durdu.

Geçen Haftanın Laboratuvarı: Nginx ve Port Yönlendirme

Gerçek bir senaryoda Nginx web sunucusunu saniyeler içinde ayağa kaldırdık.

Komutun Anatomisi

```
$ sudo docker run -d -p 8080:80 --name benim-sitem nginx
```

- `-d`: Konteyneri arka planda (detach) çalıştırır. Terminal kilitlenmez.
- `--name`: İzlemesi kolay olsun diye özel isim verdik.
- `nginx`: Docker Hub'dan çekilecek resmi imajın adıdır.

En Kritik Parametre: `-p 8080:80`

Konteynerin içindeki **80** portunu, Ubuntu Host makinemizin **8080** portuna bağladık. Dışarıdan gelen trafik içeri yönlendirildi.

Adım 4: Konteynerleri Yönetmek (Başlat / Durdur / Sil)

Uygulamamız çalışıyor. Peki buna nasıl müdahale edeceğiz?

1. Çalışanları Listeleme

```
$ sudo docker ps
```

Aktif konteynerleri ve ID'lerini listeler.

2. Konteyneri Durdurma

```
$ sudo docker stop benim-sitem
```

Durdurur. Tarayıcıda sitenin kapandığını görün.

3. Tüm Konteynerleri Görme

```
$ sudo docker ps -a
```

Durmuş olanlar dahil hepsini listeler.

4. Konteyneri Tamamen Silme

```
$ sudo docker rm benim-sitem
```

Konteyneri (çalışan kopyayı) tamamen siler.

Adım 5: Konteynerin İçine Sızmak ve Log Okumak

Konteynerler siyah kutu değildir. Geliştirme sürecinde içlerinde neler olup bittiğini görmemiz gerekir. *(Önce nginx konteynerinizi tekrar çalıştırın: `docker start benim-sitem`)*

Trafik Loglarını İncelemek

```
$ sudo docker logs benim-sitem
```

Web sitenize kimin, saat kaçta, hangi tarayıcı ile girdiğini gösteren Nginx erişim kayıtlarını anlık olarak basar.

Konteynerin İçine Girmek (exec)

```
$ sudo docker exec -it benim-sitem bash
```

Çok Kritik! Bu komut sizi konteynerin içindeki izole dosya sistemine sokar. Terminal dizininin `root@<container-id>:/#` olarak değiştiğini göreceksiniz. Çıkmak için `exit` yazın.

Kritik Kavram: Kalıcı Veri (Volumes / Bind Mounts)

Konteynerler **geçicidir (ephemeral)**. Konteyneri sildiğinizde (`docker rm`), içindeki tüm veriler (yüklenen dosyalar, veritabanı kayıtları) yok olur.

Bu sorunu çözmek ve **kendi web sitemizi yayınlamak** için, Ubuntu makinemizdeki bir klasörü konteynerin içindeki bir klasöre "Kısayol/Köprü" gibi bağlamalıyız. Buna **Volume** veya **Bind Mount** denir.

Adım 6: Kendi HTML Sayfamızı Yayınlayalım

Önce eski konteyneri silelim: `sudo docker rm -f benim-sitem`

1. Ubuntu'da bir klasör ve dosya oluşturalım

```
$ mkdir ~/sitem
$ cd ~/sitem
$ echo "<h1>Benim Ilk Docker Sitem</h1>" > index.html
```

2. Klasörü bağlayarak Nginx'i yeniden başlatalım

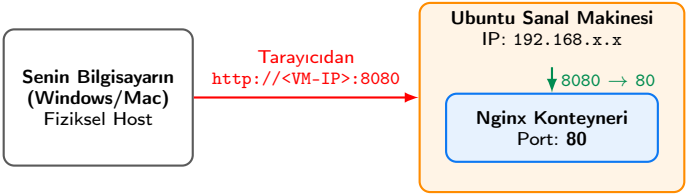
```
$ sudo docker run -d -p 8080:80 -v ~/sitem:/usr/share/nginx/html --name benim-sitem nginx
```

Test Edin

Tarayıcıdan `localhost:8080`'e tekrar girin. Artık Nginx'in varsayılan sayfasını değil, **kendi yazdığınız HTML sayfasını** göreceksiniz!

Adım 7: Dış Dünyadan (Fiziksel Makineden) Erişim Mantığı

Şu ana kadar web sitemize sadece Ubuntu VM'in içindeki tarayıcıdan (`localhost`) eriştik. Peki, laboratuvardaki **kendi asıl bilgisayarımızdan (Windows/Mac)** bu siteye nasıl gireceğiz?



Mantık: Windows üzerinde `localhost` yazarsanız, Windows *kendi* içinde web sunucusu arar ve bulamaz. Konteynere ulaşmak için **Ubuntu sanal makinesinin ağ üzerindeki IP adresini** bilmemiz ve kapısını çalmamız gerekir.

Adım 8: Ubuntu Sanal Makinesinin IP Adresini Bulma

Fiziksel bilgisayarımızdan Ubuntu'ya gidebilmek için Ubuntu'nun IP adresini öğrenmeliyiz.

Ubuntu Terminalinde IP Sorgulama

Ubuntu terminalinize aşağıdaki komutu yazın:

`$ ip a` *(veya `hostname -I`)*

- Ekrana gelen listede **enp0s3** veya **eth0** gibi bir ağ bağdaştırıcısı arayın.
- `inet` kelimesinin yanındaki adres sizin IP'nizdir. (Örn: `192.168.1.50` veya `10.0.2.15`)

Ağ Ayarı Hatırlatması

Geçen hafta gördüğümüz gibi; asıl bilgisayarınızın bu IP'ye ulaşabilmesi için VM'in ağ ayarı VirtualBox/VMware üzerinden **Bridged (Köprü)** veya **Host-Only** modunda olmalıdır.

Adım 9: Asıl Bilgisayardan (Windows/Mac) Siteyi Açma

IP adresimizi (Örn: `192.168.1.50`) öğrendik. Nginx konteynerimiz de Ubuntu üzerinde 8080 portundan yayına devam ediyor.

Windows/Mac Tarayıcısında Test

1. Ubuntu'dan çıkıp asıl Windows/Mac masaüstünüze dönün.
2. Chrome, Edge veya Safari'yi açın.
3. Adres çubuğuna şunu yazın: `http://<UBUNTU_IP_ADRESINIZ>:8080`

Örnek: `http://192.168.1.50:8080`

Tebrikler!

Ekranda **"Benim Ilk Docker Sitem"** yazısını gördüyseniz, dışarıdaki bir makineden sanal makinenize, oradan da Docker konteynerinin içine kadar uzanan başarılı bir ağ köprüsü (routing) kurdunuz demektir.

Neden Bu Kadar Önemli? (Gerçek Bulut Senaryosu)

Şu an laboratuvarda yaptığınız bu işlem, **gerçek bulut bilişim projelerinin (AWS, Azure, Google Cloud) birebir simülasyonudur.**

Bileşen	Laboratuvar Ortamı	Gerçek Bulut Ortamı (Örn: AWS)
Kullanıcı Cihazı	Laboratuvardaki Windows Bilgisayarınız	Evdeki Kişisel Bilgisayarınız
Sunucu (Host)	VirtualBox içindeki Ubuntu VM	AWS Frankfurt Veri Merkezindeki Sunucu (EC2)
IP Adresi	`192.168.x.x` (Yerel Ağ IP'si)	`3.120.x.x` (Açık/Public İnternet IP'si)
Yayınlanan Uygulama	Sizin Nginx Docker Konteyneriniz	Şirketin Gerçek Web Sitesi / API'si

Birkaç hafta sonra bulut hesaplarımızı açtığımızda; bugün VirtualBox'ta yaptığımız her şeyi, binlerce kilometre uzaktaki gerçek veri merkezlerinde yapacağız.

Kontrol Listesi ve Teslim (Çıktılar)

Öğrenci Kontrol Listesi

- `nginx` imajı ile port yönlendirmesi (`-p`) yapılarak sunucu ayağa kaldırıldı.
- `docker logs` ile erişim kayıtları incelendi.
- `docker exec -it ... bash` ile konteynerin dosya sistemine giriş yapıldı ve `exit` ile çıkıldı.
- `-v` (Volume) parametresi kullanılarak Ubuntu'daki `index.html` dosyası Nginx konteynerine bağlandı.

Teslim Edilecekler (Ödev)

- **2 Ekran Görüntüsü:**
  1. `docker exec` ile konteynerin içine (`root@...`) girildiğini gösteren terminal ekranı.
  2. Firefox'ta görünen **kendi yazdığınız ("Benim Ilk Docker Sitem")** web sayfasının görüntüsü.
- **Kısa Cevap:** Volume (Bind Mount) mantığı bize uygulama geliştirirken nasıl bir kolaylık sağlar?

Laboratuvar Sonu - Sorularınız?